Nama : Elsadanika Maura Sitorus

NIM : 211344013 – 2NK

RANGKUMAN MINGGU 3 MIKROPROSESOR & INTERFACE

Overflow merupakan kondisi ketika hasil operasi mengalami over capacity terhadap register atau memory areanya. Overflow dilihat dari range data. Berikut ini adalah range data 8 bit:

Data 8 bit tidak bertanda	Data 8 bit bertanda
0 - 255	(-128) – (+127)

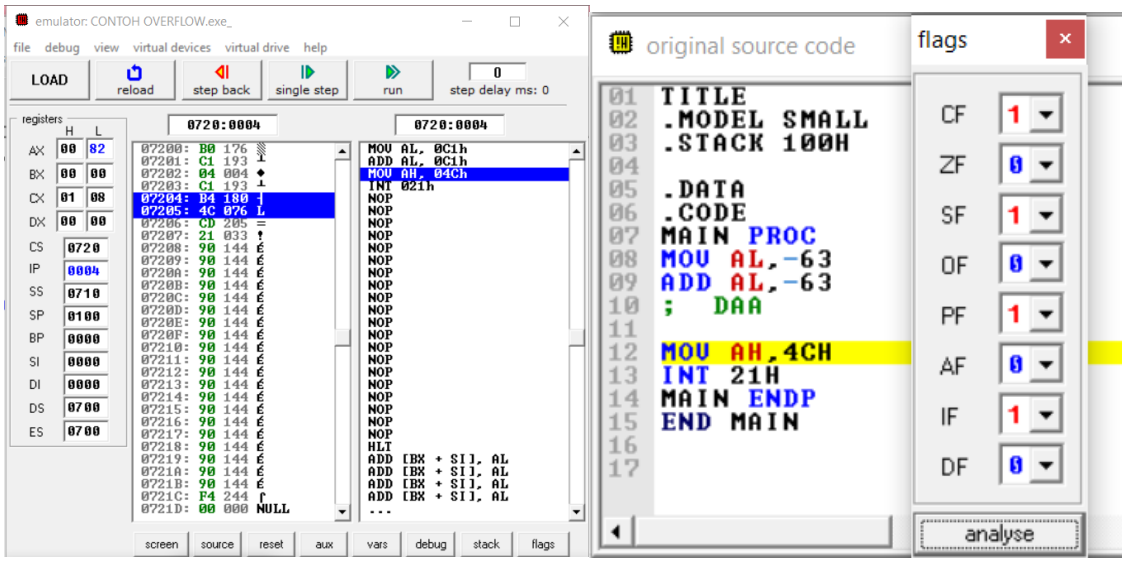
Dalam penjumlahan decimal, overflow diketahui ketika hasil penjumlahan melebihi 8 bit, sehingga satu memory area tidak dapat menampungnya dan memerlukan area lebih. Berikut adalah contoh penjumlahan decimal dengan kondisi overflow.

Data 1	Data 2	Hasil	Overflow
79	02	81	00
79	79	58	01

Untuk mengetahui suatu penjumlahan decimal mengalami overflow maka, bilangan diubah ke bentuk biner komplemen 2 terlebih dahulu lalu diubah ke heksadesimal pada simulator. Berikut ini adalah contoh penjumlahan bilangan decimal yang tidak mengalami overflow pada simulator.

$-63_{(10)} = 1\ 0111111_{(2SM)} \rightarrow 1\ 1000000_{(K'1)} \rightarrow 1\ 1000001_{(K'2)} \rightarrow C1_{(16)}$

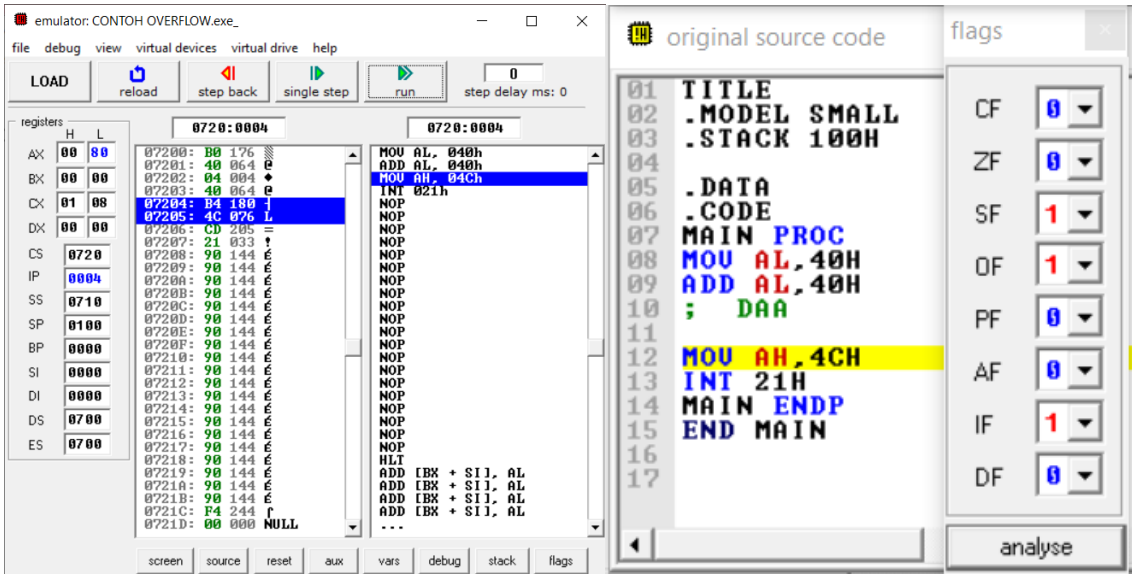
(-63) + (-63) = (-126)



Berikut ini adalah contoh penjumlahan bilangan hexadecimal yang mengalami overflow pada simulator.

$40H_{(16)} = 0\ 1000000_{(2)} \rightarrow 64_{(10)}$

64 + 64 = 128



Untuk mendeteksi overflow, dilakukan pada penjumlahan heksadesimal. Ketika OF dideteksi saat decimal maka nilai ZF = 1 (saat operasi = 0) tetapi instruksi transfer data tidak memengaruhi ZF karena zero flag hanya berpengaruh di ALU. Prosesor INTEL 8086 selalu menggunakan komplemen 2 atau bilangan bertanda. Untuk menggunakan perintah Jump, dibedakan menjadi dua, yaitu:

1. Signed Number
   JA & JB
2. Unsigned Number
   JG & JL

PENJUMLAHAN BILANGAN OKTAL

Bilangan Oktal = 8^3 8^2 8^1 8^0

DESIMAL	BINER	OKTAL	HEXA
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Cara konversi decimal ke oktal adalah sebagai berikut

$4327_{(8)}$ → …$_{(10)}$		
$7 . 8^0$	= 7 . 1 →	7
$2 . 8^1$	= 2 . 8 →	16
$3 . 8^2$	= 3 . 64 →	192
$4 . 8^3$	= 4 . 512 →	2048
---------- +		---------- +
$4327_{(8)}$ =		$2263_{(10)}$

Cara konversi heksadesimal ke oktal adalah dengan menambahkan 8 jika uji satuan bernilai >7. Contohnya adalah sebagai berikut.

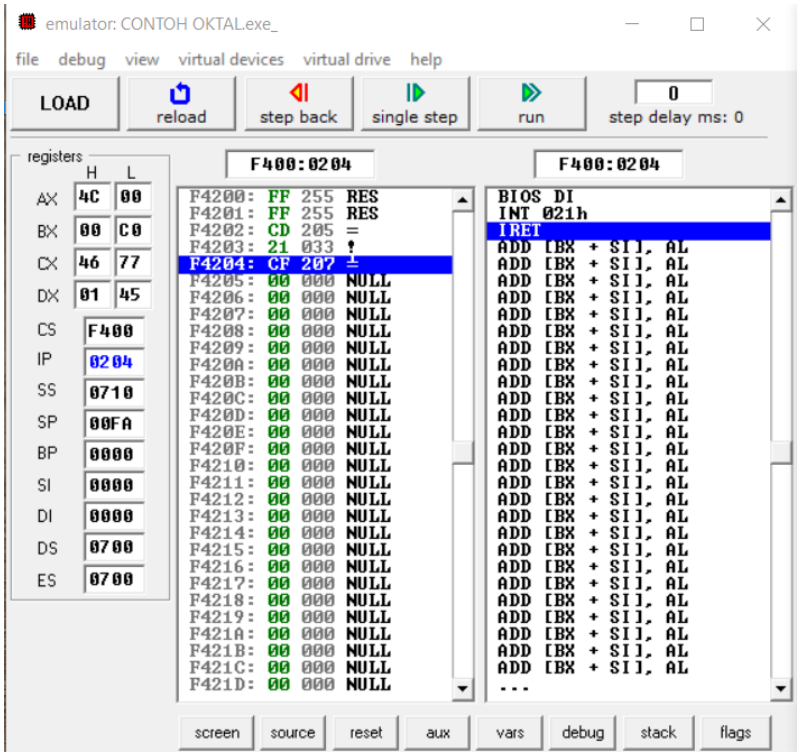
77	
46	
------- +	
BD	Nilai D > 7
8 8	
------- +	
145	

Bilangan oktal memiliki satuan, delapanan, enam puluh empatan, dan selanjutnya. Untuk memisahkan satuan dan delapan, digunakan perintah logic AND dengan symbol Λ.
Contoh untuk memisahkan BD antara satuan dengan delapanannya adalah sebagai berikut.
1011 1101
0000 1111 Λ → AND dengan 00FH
---------- +
0000 1101 = 0D → dapat dilakukan proses CMP 07H

1011 1101
1111 0000 Λ → AND dengan 0F0H
---------- +
1011 0000 = B0 → dapat dilakukan proses CMP 70H
Berikut ini contoh program menjumlahkan 77+46 (heksadesimal) = 145 (oktal) dengan hasil oktal yang disimpan pada register DX.

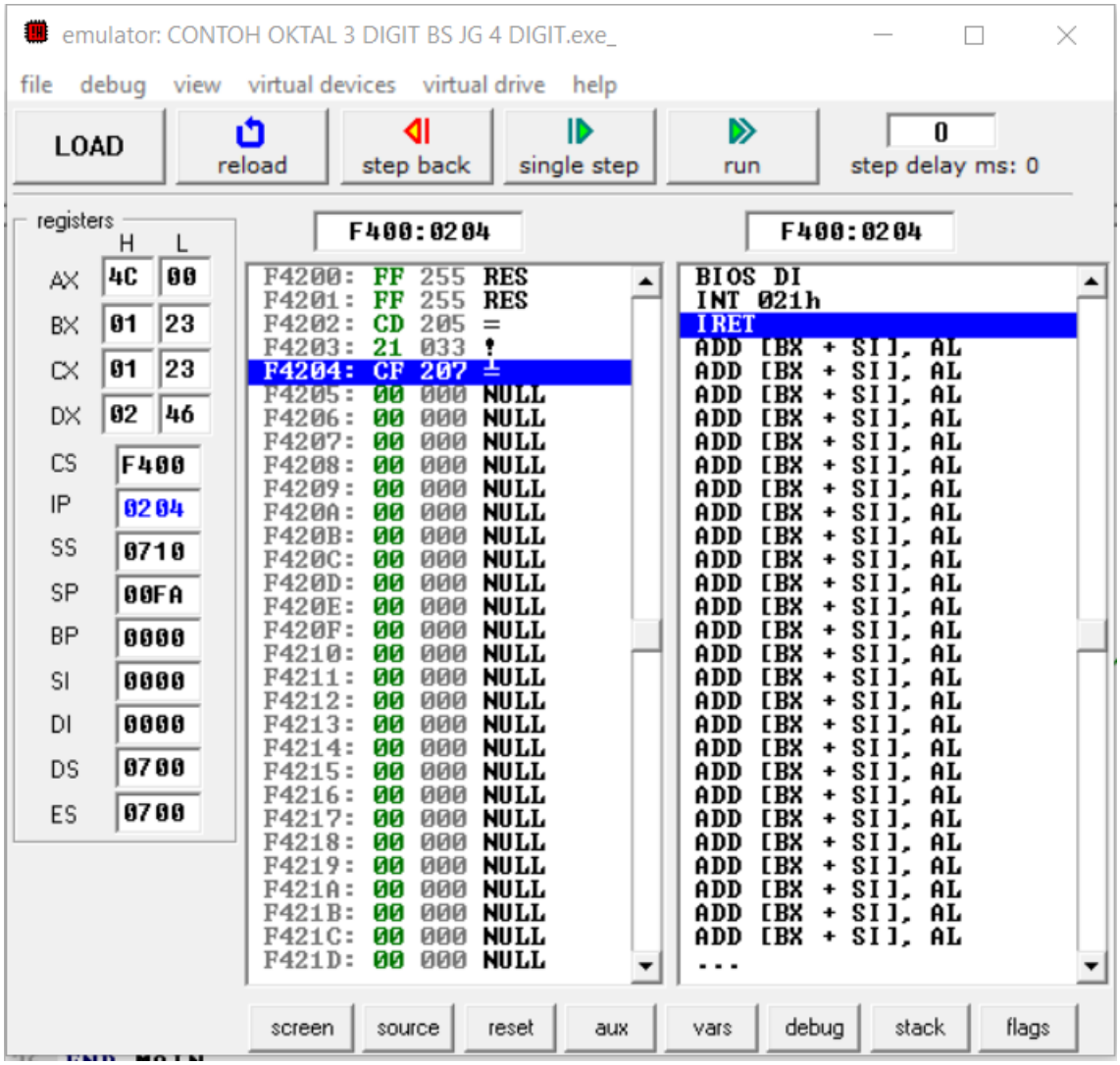
original source code

```
01 TITLE PENJ OKTAL
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 .CODE
06 MAIN PROC
07 MOV DX,0
08 MOV DL,CL
09 ADD DL,CH
10 MOV BL,DL
11 AND BL,0FH   ;SAAT DIUJI
12 CMP BL,7
13 JBE L1       ;SATUAN SUDAH SELESAI DIUJI
14
15 ADD DL,8     ;KOREKSI
16 L1:  MOV BL,DL    ;HARUS DIPINDAH SUPAYA DATA DI DL GA RUSAK/HILANG
17 AND BL,0F0H
18 CMP BL,70H
19 JBE L2
20 ADD DX,80H   ;KOREKSI
21 ;ADC DH,0
22
23 L2:  MOV AH,4CH
24 INT 21H
25 MAIN ENDP
26 END MAIN
```



Berikut ini adalah contoh penjumlahan oktal 3 digit, data : 0123 + 0123 = 0246 (oktal).

```
01 TITLE PENJ OKTAL 3 DIGIT
02 .MODEL SMALL
03 .STACK 100H
04
05 .DATA
06 .CODE
07 MAIN PROC
08 MOV DX,0
09 MOV DX,CX    ;UJI 1
10 ADD DX,BX
11
12 MOV AX,DX
13 AND AX,000FH   ;SAAT DIUJI
14 CMP AX,0007H
15 JBE L0       ;SATUAN SUDAH SELESAI DIUJI
16
17 ADD DX,0008H     ;KOREKSI
18
19 L0:  MOV AX,DX    ;HARUS DIPINDAH SUPAYA DATA DI DL GA RUSAK/HILANG
20 AND AX,00F0H
21 CMP AX,0070H
22 JBE L1
23
24 ADD DX,0080H   ;KOREKSI
25
26 L1:  MOV AX,DX
27 AND AX,0F00H
28 CMP AX,0700H
29 JBE L2
30
31 ADD DX,0800H
32
33 L2:  MOV AH,4CH
34 INT 21H
35 MAIN ENDP
36 END MAIN
```


emulator: CONTOH OKTAL 3 DIGIT BS JG 4 DIGIT.exe_
file debug view virtual devices virtual drive help
LOAD
reload
step back
single step
run
0
step delay ms: 0
registers
H L
AX 4C 00
BX 01 23
CX 01 23
DX 02 46
CS F400
IP 0204
SS 0710
SP 00FA
BP 0000
SI 0000
DI 0000
DS 0700
ES 0700
F400:0204
F4200: FF 255 RES
F4201: FF 255 RES
F4202: CD 205 =
F4203: 21 033 !
F4204: CF 207 ≟
F4205: 00 000 NULL
F4206: 00 000 NULL
F4207: 00 000 NULL
F4208: 00 000 NULL
F4209: 00 000 NULL
F420A: 00 000 NULL
F420B: 00 000 NULL
F420C: 00 000 NULL
F420D: 00 000 NULL
F420E: 00 000 NULL
F420F: 00 000 NULL
F4210: 00 000 NULL
F4211: 00 000 NULL
F4212: 00 000 NULL
F4213: 00 000 NULL
F4214: 00 000 NULL
F4215: 00 000 NULL
F4216: 00 000 NULL
F4217: 00 000 NULL
F4218: 00 000 NULL
F4219: 00 000 NULL
F421A: 00 000 NULL
F421B: 00 000 NULL
F421C: 00 000 NULL
F421D: 00 000 NULL
F400:0204
BIOS DI
INT 021h
IRET
ADD [BX + SI], AL
...
screen source reset aux vars debug stack flags